

Assignment 1: Basic Interaction with Large Language Models (LLMs)

Course: Agentic AI & Large Language Models

Assignment No.: 1

Title: Basic Interaction with Large Language Models Using API

1. Objective

The objective of this assignment is to understand how Large Language Models (LLMs) work through APIs. Students will learn to:

- Connect to an LLM using an API.
 - Send prompts to the model.
 - Receive and display responses.
 - Perform multiple NLP tasks.
 - Understand prompt engineering.
 - Compare responses generated using different prompts.
-

2. Problem Statement

Develop a simple application that interacts with a Large Language Model (LLM) using an API.

The application should:

- Accept user input.
 - Send the input to an LLM.
 - Display the generated response.
 - Support multiple NLP tasks:
 - Question Answering
 - Text Summarization
 - Translation
 - Compare outputs generated using different prompts.
 - Document the observations regarding prompt effectiveness.
-

3. Learning Outcomes

After completing this assignment, students will be able to

- Understand API-based interaction with LLMs.

- Use prompt engineering techniques.
 - Compare different prompting strategies.
 - Evaluate generated responses.
 - Build the foundation for Agentic AI applications.
-

4. Software Requirements

Students may use any one of the following.

Programming Language

- Python (Recommended)

Framework (Optional)

- Streamlit
- Flask
- Gradio

IDE

- VS Code
- PyCharm
- Google Colab
- Jupyter Notebook

LLM APIs

Choose any ONE

- OpenAI GPT
- Google Gemini API
- Anthropic Claude API
- Groq API
- Together AI
- HuggingFace Inference API

5. Create Your Project Folder

Create a folder named: Assignment1_LLM

Inside it, create:

```
Assignment1_LLM
├── app.py
├── requirements.txt
└── README.md
```

6. Create a Virtual Environment

A virtual environment keeps project dependencies isolated.

Run: `python -m venv venv`

This creates:

```
Assignment1_LLM
├── venv
└── app.py
```

7. Activate the Virtual Environment

Windows

`venv\Scripts\activate`

If successful, you'll see:

```
(venv)
C:\Assignment1_LLM>
```

The (venv) prefix means the environment is active.

8. Install Required Libraries

We'll use **Google Gemini** because it has a generous free tier for learning.

Install the SDK:

```
pip install google-genai
```

Also install **python-dotenv** to manage your API key securely:

```
pip install python-dotenv
```

Save installed packages:

```
pip freeze > requirements.txt
```

9. Get a Free Gemini API Key

Go to the official Google AI Studio website and sign in with your Google account.

1. Open Google AI Studio.
2. Create a new API key.
3. Copy the key.

It will look something like:

```
AIzaSyXXXXXXXXXXXXX...
```

Keep it private.

Create a .env File

In your project folder, create a file named:

```
.env
```

Add:

```
GEMINI_API_KEY=YOUR_API_KEY_HERE
```

Replace YOUR_API_KEY_HERE with your actual key.

10. Create app.py

Start with the following code:

```
import os
from dotenv import load_dotenv
from google import genai
```

```
# Load environment variables
load_dotenv()
```

```
# Read API key
```

```
api_key = os.getenv("GEMINI_API_KEY")
```

```
# Create client
```

```
client = genai.Client(api_key=api_key)
```

```
print("Connected Successfully!")
```

Run it:

```
python app.py
```

If everything is configured correctly, you'll see:

Connected Successfully!

11. Send Your First Prompt

Replace the contents of app.py with:

```
import os
```

```
from dotenv import load_dotenv
```

```
from google import genai
```

```
load_dotenv()
```

```
client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))
```

```
response = client.models.generate_content(  
    model="gemini-2.5-flash",  
    contents="What is Artificial Intelligence?"  
)
```

```
print(response.text)
```

Run:

```
python app.py
```

Example output:

Artificial Intelligence (AI) is the simulation of human intelligence by machines...

12. Accept User Input

Now make it interactive:

```
import os
```

```

from dotenv import load_dotenv
from google import genai

load_dotenv()

client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

prompt = input("Enter your question: ")

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt
)

print("\nResponse:\n")
print(response.text)

```

Sample interaction:

Enter your question:
Explain Machine Learning.

Response:
Machine Learning is...

13. Add a Simple Menu

```

print("===== LLM APPLICATION =====")
print("1. Question Answering")
print("2. Summarization")
print("3. Translation")

choice = input("Choose an option: ")

```

14. Based on the user's choice, you can prepend instructions to the prompt.

Handle Different Tasks

```

text = input("Enter text: ")

if choice == "1":
    prompt = text

elif choice == "2":
    prompt = f"Summarize the following text:\n\n{text}"

elif choice == "3":
    prompt = f"Translate the following into Marathi:\n\n{text}"

```

```
else:
    print("Invalid choice")
    exit()
```

Then send prompt to the model as before.

15. Complete app.py

```
import os
from dotenv import load_dotenv
from google import genai

# -----
# Load API Key
# -----
load_dotenv()

api_key = os.getenv("GEMINI_API_KEY")

if not api_key:
    print("Error: GEMINI_API_KEY not found.")
    exit()

client = genai.Client(api_key=api_key)

# -----
# Function to send prompt to Gemini
# -----
def ask_gemini(prompt):
    try:
        response = client.models.generate_content(
            model="gemini-2.5-flash",
            contents=prompt
        )
        return response.text

    except Exception as e:
        return f"Error: {e}"

# -----
# Main Program
# -----
while True:

    print("\n")
    print("=" * 55)
    print("    BASIC LLM APPLICATION")
    print("=" * 55)

    print("1. Question Answering")
```

```

print("2. Text Summarization")
print("3. Translation")
print("4. Exit")

print("-" * 55)

choice = input("Choose an option (1-4): ")

# -----
# Question Answering
# -----
if choice == "1":

    question = input("\nEnter your question:\n> ")

    prompt = f"""
You are an expert AI assistant.

Answer the following question clearly and accurately.

Question:
{question}
"""

    answer = ask_gemini(prompt)

    print("\nGenerated Answer")
    print("-" * 55)
    print(answer)

# -----
# Summarization
# -----
elif choice == "2":

    text = input("\nPaste the paragraph to summarize:\n> ")

    prompt = f"""
Summarize the following text in 5 concise bullet points.

Text:
{text}
"""

    answer = ask_gemini(prompt)

    print("\nSummary")
    print("-" * 55)
    print(answer)

```

```

# -----
# Translation
# -----
elif choice == "3":

    language = input("\nTranslate to which language? ")

    text = input("\nEnter text:\n> ")

    prompt = f"""
Translate the following text into {language}.

Text:
{text}

Only provide the translated text.
"""

    answer = ask_gemini(prompt)

    print("\nTranslated Text")
    print("-" * 55)
    print(answer)

# -----
# Exit
# -----
elif choice == "4":

    print("\nThank you for using the Basic LLM Application.")
    break

else:

    print("\nInvalid choice! Please enter 1-4.")

input("\nPress Enter to continue...")

```

Sample Output

```

=====
          BASIC LLM APPLICATION
=====

```

1. Question Answering
2. Text Summarization
3. Translation
4. Exit

Choose an option (1-4): 1

Enter your question:

> What is Machine Learning?

Generated Answer

Machine Learning is a branch of Artificial Intelligence...

Translation Example

Choose an option: 3

Translate to which language?

Marathi

Enter text:

Artificial Intelligence is changing the world.

Translated Text

कृत्रिम बुद्धिमत्ता जग बदलत आहे.

Summarization Example

Choose an option: 2

Paste the paragraph:

Large Language Models are transforming...

Summary

-
- LLMs process natural language.
 - They use transformer architecture.
 - They are trained on large datasets.
 - They support many NLP tasks.
 - Prompt quality affects responses.